

QUIZ 3 Linda Miao

Question 1. Write a sql script that creates a database of any tables, insert any number of rows and then write query such that the query optimizer chooses a parallel query plan to execute this query. Capture the query plan. Modify the query in any way you like so that the modified query results in a serial query plan. You are allowed to minimally change the query. Please do not change the DB schema or the rows in the DB. Capture the new query plan. Explain why the query optimizer switched from a parallel plan to a serial plan after the modification. Please submit:

- script that creates DB and inserts rows
- The query
- The query plan
- The query after modification
- The new query plan
- Explanation of why the query plan was changed from a parallel plan to a serial plan.

-- Topic: Parallel vs Serial Query Plans

-- Database: PARALLELQUIZ3

-- Tables: Agents, Houses, Sales (Zillow-style real estate data)

CREATE DATABASE PARALLELQUIZ3;

GO

USE PARALLELQUIZ3;

GO

-- clean up old tables

IF OBJECT_ID('Sales', 'U') IS NOT NULL DROP TABLE Sales;

IF OBJECT_ID('Houses', 'U') IS NOT NULL DROP TABLE Houses;

IF OBJECT_ID('Agents', 'U') IS NOT NULL DROP TABLE Agents;

GO

-- create table-agents

```
CREATE TABLE Agents (  
    AgentID INT PRIMARY KEY,  
    AgentName VARCHAR(100),  
    Agency VARCHAR(100),  
    City VARCHAR(50)  
);  
GO
```

-- create table - houses

```
CREATE TABLE Houses (  
    PropertyID INT PRIMARY KEY,  
    Address VARCHAR(150),  
    City VARCHAR(50),  
    State VARCHAR(20),  
    Price DECIMAL(10,2),  
    Bedrooms INT,  
    Bathrooms INT,  
    SqFt INT  
);  
GO
```

-- create table - sales

```
CREATE TABLE Sales (  
    SaleID INT PRIMARY KEY,  
    PropertyID INT,  
    AgentID INT,
```

```
SalePrice DECIMAL(10,2),
SaleDate DATE
);
GO
```

```
-- insert rows - agents(100 rows) by loop; NO execution plan ON when insert
```

```
-- Each agent gets a name, agency, and city based on math patterns.
```

```
DECLARE @j INT = 1;
WHILE @j <= 100
BEGIN
    INSERT INTO Agents VALUES (
        @j,
        'Agent_' + CAST(@j AS VARCHAR(10)),
        'Agency_' + CAST((@j % 10) AS VARCHAR(10)),
        CASE (@j % 5)
            WHEN 0 THEN 'Seattle'
            WHEN 1 THEN 'Portland'
            WHEN 2 THEN 'San Francisco'
            WHEN 3 THEN 'Los Angeles'
            ELSE 'Las Vegas' END
    );
    SET @j = @j + 1;
END
GO
```

```
-- insert rows - houses (3000 rows) by loop; NO execution plan ON when insert
```

```
-- insert 3,000 houses with addresses, cities, prices,
```

```

-- bedrooms, bathrooms, and square footage.
-- Prices increase with each row starting at $200,100.
DECLARE @i INT = 1;
WHILE @i <= 3000
BEGIN
    INSERT INTO Houses VALUES (
        @i,
        CAST(@i AS VARCHAR(10)) + ' Main St',
        CASE (@i % 5)
            WHEN 0 THEN 'Seattle'
            WHEN 1 THEN 'Portland'
            WHEN 2 THEN 'San Francisco'
            WHEN 3 THEN 'Los Angeles'
            ELSE 'Las Vegas' END,
        CASE (@i % 5)
            WHEN 0 THEN 'WA'
            WHEN 1 THEN 'OR'
            WHEN 2 THEN 'CA'
            WHEN 3 THEN 'CA'
            ELSE 'NV' END,
        CAST(200000 + (@i * 100) AS DECIMAL(10,2)),
        (@i % 5) + 1,
        (@i % 3) + 1,
        800 + (@i % 3000)
    );
    SET @i = @i + 1;
END

```

GO

-- insert rows - sales (5000 rows); NO execution plan ON when insert

-- insert 5,000 sale records linking houses to agents.

-- Sale prices are slightly higher than listing prices.

-- Sale dates are spread across the year 2023.

DECLARE @k INT = 1;

WHILE @k <= 5000

BEGIN

INSERT INTO Sales VALUES (

 @k,

 (@k % 3000) + 1,

 (@k % 100) + 1,

 CAST(200000 + (@k * 150) AS DECIMAL(10,2)),

 DATEADD(DAY, @k % 365, '2023-01-01')

);

SET @k = @k + 1;

END

GO

-- VERIFY ROW COUNTS

-- Check that all 3 tables have the correct number of rows.

-- Expected: Agents=100, Houses=3000, Sales=5000

SELECT 'Agents' AS TableName, COUNT(*) AS TotalRows FROM Agents

UNION ALL

SELECT 'Houses', COUNT(*) FROM Houses

UNION ALL

```
SELECT 'Sales',          COUNT(*)      FROM Sales;
GO
```

```
-- UPDATE STATISTICS
```

```
-- Tell the optimizer how many rows and what data exists.
```

```
-- This helps the optimizer make better decisions about
```

```
-- whether to use a parallel or serial plan.
```

```
UPDATE STATISTICS Agents;
```

```
UPDATE STATISTICS Houses;
```

```
UPDATE STATISTICS Sales;
```

```
GO
```

```
-- Try find a parallel query(but it only output serial plan)
```

```
-- Ctrl+M ON before running this block.
```

```
SELECT
```

```
    h.City,
```

```
    h.State,
```

```
    COUNT(s.SaleID)    AS TotalSales,
```

```
    AVG(s.SalePrice)   AS AvgSalePrice,
```

```
    MAX(s.SalePrice)   AS MaxSalePrice,
```

```
    MIN(s.SalePrice)   AS MinSalePrice,
```

```
    AVG(CAST(h.SqFt AS DECIMAL(10,2))) AS AvgSqFt
```

```
FROM Houses    h
```

```
JOIN Sales    s ON h.PropertyID = s.PropertyID
```

```
JOIN Agents   a ON s.AgentID    = a.AgentID
```

```
WHERE h.Price > 200000
```

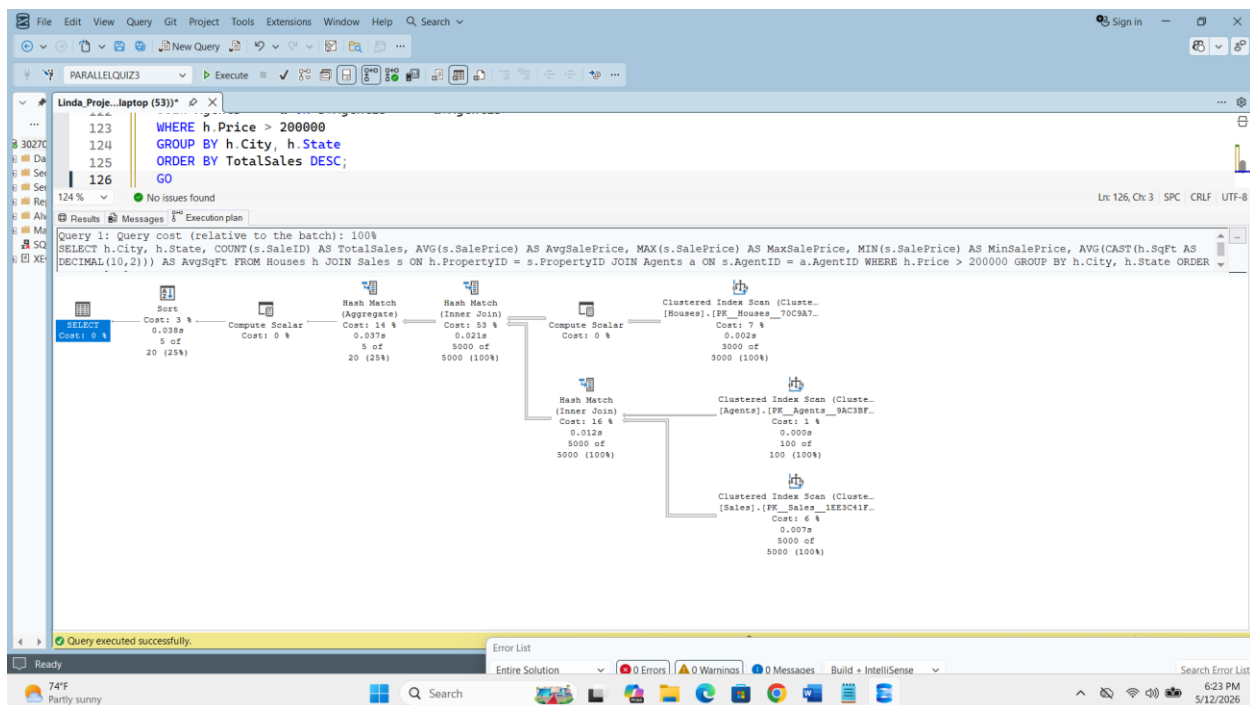
```
GROUP BY h.City, h.State
```

ORDER BY TotalSales DESC;

GO

EXPLANATION:

My initial query ran without any hint. The query joined 3 tables with 5 aggregate functions (COUNT, AVG, MAX, MIN, and AVG of SqFt), but the total number of rows processed across the 3 tables (100 Agents + 3,000 Houses + 5,000 Sales = 8,100 rows) was not large enough to push the estimated query cost above SQL Server's default Cost Threshold for Parallelism of 5 cost units. As a result, the optimizer automatically chose a serial plan



-- force parallel; Ctrl+M ON

-- This query joins all 3 tables and computes 5 aggregates.

-- The hint ENABLE_PARALLEL_PLAN_PREFERENCE tells the optimizer:

-- "prefer a parallel plan if one is available."

-- The optimizer responds with Gather Streams and

-- Repartition Streams operators = multiple CPU threads working.

SELECT

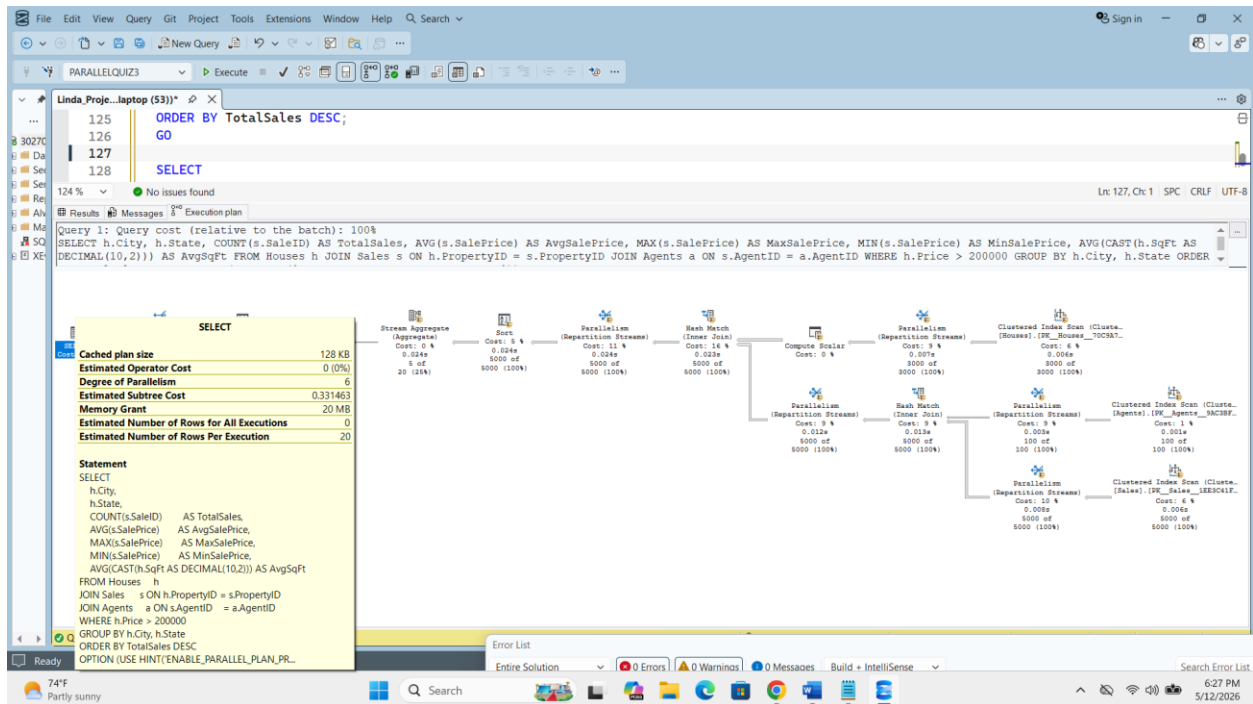
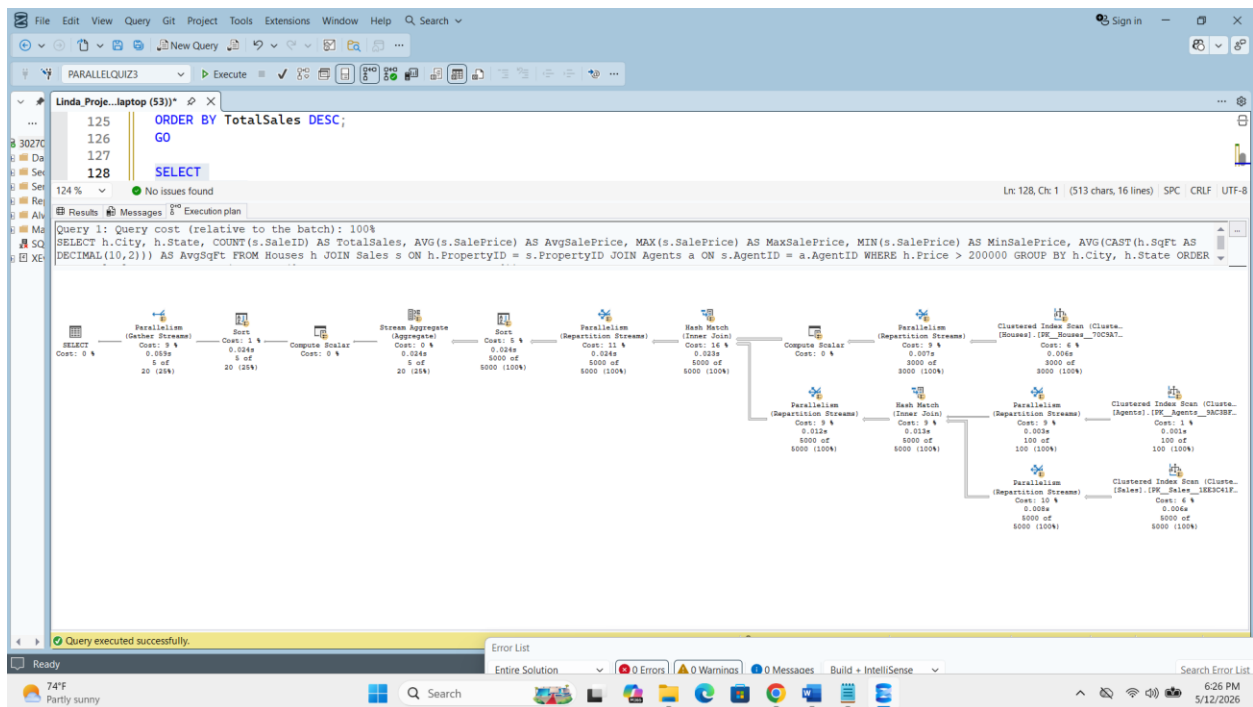
```

h.City,
h.State,
COUNT(s.SaleID)    AS TotalSales,
AVG(s.SalePrice)    AS AvgSalePrice,
MAX(s.SalePrice)    AS MaxSalePrice,
MIN(s.SalePrice)    AS MinSalePrice,
AVG(CAST(h.SqFt AS DECIMAL(10,2))) AS AvgSqFt
FROM Houses    h
JOIN Sales    s ON h.PropertyID = s.PropertyID
JOIN Agents    a ON s.AgentID    = a.AgentID
WHERE h.Price > 200000
GROUP BY h.City, h.State
ORDER BY TotalSales DESC
OPTION (USE HINT('ENABLE_PARALLEL_PLAN_PREFERENCE'));
GO

```

EXPLANATION:

In order to achieve a parallel plan, I applied the hint `OPTION (USE HINT('ENABLE_PARALLEL_PLAN_PREFERENCE'))`. This hint instructs the optimizer to prefer a parallel plan when one is available. Without this hint, the optimizer chose a serial plan because the query cost did not exceed the Cost Threshold for Parallelism. The hint overrides this threshold, telling the optimizer to prefer parallel execution regardless of cost. This is why the same query — with the same tables and same rows — produced a parallel plan this time. The execution plan confirms it — it displays two Parallelism operators: Gather Streams and Repartition Streams. These operators allow multiple CPU threads to work simultaneously, splitting and merging the query work across threads at the same time. Another way to achieve a parallel plan without a hint would be to increase the number of rows significantly — for example, inserting 50,000 or more rows into Houses and 100,000 rows into Sales. However, based on my laptop's processing capacity, inserting that many rows would cause SSMS to crash. Therefore, using the hint was the safest and most practical solution.



-- SERIAL QUERY (minimal change = MAXDOP 1)

-- Ctrl+M ON before running this block.

--

-- ONLY ONE THING CHANGED: the hint at the bottom.
-- OPTION (MAXDOP 1) means Maximum Degree of Parallelism = 1.
-- This is a hard rule: SQL Server can only use 1 CPU thread.
-- The optimizer cannot use Gather Streams or Repartition Streams.
-- Result: a simpler serial plan with no Parallelism operators.

```
SELECT
    h.City,
    h.State,
    COUNT(s.SaleID)    AS TotalSales,
    AVG(s.SalePrice)   AS AvgSalePrice,
    MAX(s.SalePrice)   AS MaxSalePrice,
    MIN(s.SalePrice)   AS MinSalePrice,
    AVG(CAST(h.SqFt AS DECIMAL(10,2))) AS AvgSqFt
FROM Houses    h
JOIN Sales     s ON h.PropertyID = s.PropertyID
JOIN Agents    a ON s.AgentID    = a.AgentID
WHERE h.Price > 200000
GROUP BY h.City, h.State
ORDER BY TotalSales DESC
OPTION (MAXDOP 1);
GO
```

EXPLANATION:

This query switched from a parallel plan to a serial plan by replacing the parallel hint with `OPTION (MAXDOP 1)`. This is the only change made to the query. `MAXDOP` stands for Maximum Degree of Parallelism. Setting it to 1 means all the work that was previously split across multiple CPU threads is now forced onto one single thread. This makes the serial plan cost higher than the parallel plan because one thread must do everything alone, while the parallel plan had multiple threads sharing the work simultaneously. The execution plan confirms it — the

screenshot below shows no Parallelism operators anywhere in the plan. All operators run sequentially on a single thread.

It is worth noting that MAXDOP can be set to other values. If MAXDOP is set to 2, 4, or 8, SQL Server is allowed to use that many threads — which may still produce a parallel plan. Our goal was a serial plan, so MAXDOP 1 was the correct choice because it is the only value that guarantees a single thread. If MAXDOP is set to 0, it means no limit at all — SQL Server decides freely how many threads to use based on the available CPUs and query cost. In our case, MAXDOP 0 would likely produce a parallel plan again since the optimizer would have full freedom to choose.

The top screenshot shows a SQL query in the query editor:

```

SELECT h.City, h.State, COUNT(s.SaleID) AS TotalSales, AVG(s.SalePrice) AS AvgSalePrice, MAX(s.SalePrice) AS MaxSalePrice, MIN(s.SalePrice) AS MinSalePrice, AVG(CAST(h.SqFt AS DECIMAL(10,2))) AS AvgSqFt
FROM Houses h
JOIN Sales s ON h.PropertyID = s.PropertyID
JOIN Agents a ON s.AgentID = a.AgentID
WHERE h.Price > 200000
GROUP BY h.City, h.State

```

The query plan shows a 'Parallel' execution mode. The plan includes a 'Hash Match (Inner Join)' operator, a 'Compute Scalar' operator, and a 'Clustered Index Scan' operator. The plan is marked as 'Parallel'.

The bottom screenshot shows the same query plan, but the execution mode has changed to 'Serial'. The plan is now marked as 'Serial'.

The 'Query Plan Statistics' window in the bottom screenshot shows the following information:

Statistic	Value
Cached plan size	120 KB
Estimated Operator Cost	0.070
Degree of Parallelism	1
Estimated Subtree Cost	0.368443
Memory Grant	3456 KB
Estimated Number of Rows for All Executions	0
Estimated Number of Rows Per Execution	20

EXPLANATION: Why the Query Plan Changed from Parallel to Serial

SQL Server's query optimizer evaluates every query and estimates its cost in optimizer cost units. When the estimated cost exceeds the server's Cost Threshold for Parallelism (default value: 5

cost units), the optimizer considers using multiple CPU threads to execute the query faster. This is called a parallel execution plan.

In this assignment, my initial query joined 3 tables (Agents, Houses, and Sales) with 5 aggregate functions (COUNT, AVG, MAX, MIN, and AVG of SqFt). However, the total number of rows — 8,100 rows across all 3 tables — was not large enough for the optimizer to naturally choose a parallel plan. The estimated cost stayed below 5 cost units, so the optimizer chose a serial plan by default. To force a parallel plan, I applied the query hint `OPTION (USE HINT('ENABLE_PARALLEL_PLAN_PREFERENCE'))`, which instructs the optimizer to prefer parallel execution regardless of cost. The resulting plan showed Parallelism operators — Gather Streams and Repartition Streams — confirming that multiple CPU threads were used.

To switch back to a serial plan, I made only one change to the query, I replaced the parallel hint with `OPTION (MAXDOP 1)`. MAXDOP stands for Maximum Degree of Parallelism. Setting it to 1 is a hard rule that forces SQL Server to use exactly one CPU thread. The optimizer cannot introduce any Parallelism operators when MAXDOP is 1, regardless of the query cost or data size. As a result, the execution plan showed no Gather Streams or Repartition Streams operators — confirming a fully serial execution plan."